Vocabulary Catches, Structure Calms: Decomposing Action Policies for AI Coding Agents on Held-Out Benchmarks and Real Traffic

Muhammad Usama
COMSATS University Islamabad
Islamabad, Pakistan
muhammadusamahoyrr@gmail.com

Awais Khalid
FAST National University of Computer and Emerging Sciences
Islamabad, Pakistan
2k7awaiskhalid@gmail.com

***Abstract*—AI coding agents run shell commands with the developer's privileges, usually behind policies that match the command text. We hold one policy engine fixed and vary only what its rules see: text, in substring and four regular-expression policies, or three facts (destructive, secret access, opaque) derived from a parsed shell syntax tree. On an independent held-out benchmark of 150 actions, fact-based rules stop 90.0% of harmful actions at a 5.7% false-positive rate, against 16.3% for the substring policy. Most of that gain is vocabulary: a regex given the analyzer's word lists comes close, and the benchmark cannot separate the two. The syntax tree shows in interruptions. On our own shell traffic, a line tokenizer with the same semantics interrupts more than twice as often as facts. In a pre-specified test on 417 other developers in the public SWE-chat corpus, facts interrupt 1.5 points less than the tokenizer per developer (macro-averaged; median 0.5), and less for 251 developers against 28. Against the vocabulary regex the typical developer's difference is near zero, and a regex tuned on our traffic matches facts. The pre-specified audit of whether stops are warranted was inconclusive; all audits were labeled by Claude agents, not humans. In exploratory agent audits, most tokenizer-only stops were benign, but a few were credential reads facts allow; regex-only stops were mostly mentions of credentials, and fact-only stops mostly code a careful reader, not the analyzer, can resolve. We release the analyzer, benchmarks, harness and a human labeling kit.**

***Index Terms*—AI coding agents, LLM agent security, guardrails, access control policies, shell command analysis, static analysis, usable security**

I. INTRODUCTION

Coding agents such as Claude Code, Cursor and Codex execute shell commands on developer machines. When they go wrong the damage is concrete: in July 2025 an agent deleted a production database despite the user's explicit code freeze, according to the user [1]; malicious code instructing an agent to wipe local files and cloud resources was shipped inside an IDE extension, though it failed to run [2]; and a supply-chain attack invoked installed agent CLIs with permission checks disabled to collect `.env` files and SSH keys [3]. Benchmarks find that agents act unsafely in a large share of risky situations [4]–[6].

The common mitigation is an *action policy*: rules that allow, deny, or escalate each tool call. Shell commands reach these rules as strings. Agent permission systems may split compound commands and strip wrappers, but the rule itself is a pattern over the command text. Vendor documentation warns that "Bash permission patterns that try to constrain command arguments are fragile" and that a deny rule "isn't a security boundary around the program" [7]; documented hook examples grep the command for `rm -rf` [8]. Industry reports show such guards bypassed by quoting, subshells and encoding [9], [10].

A policy can fail in two directions. Too weak, it misses a destructive command phrased differently (`rm -fr`, `find -delete`, `bash -c '...'`). Too strong, it interrupts so often that approvals become reflexive: habituation to security warnings is well documented [11]–[13], and agent vendors cite approval fatigue as a reason to reduce prompts [14], [15]. We therefore ask:

RQ1 How much do text-based policies miss on independently written actions?
RQ2 What does a fact layer add, and how much of it is vocabulary, normalization, the syntax tree, and per-tool semantics?
RQ3 What does each policy cost in interruptions on our own agent traffic, and are those interruptions warranted?
RQ4 Do these costs and audit results hold, pre-specified, on other developers' traffic and on a second agent?
RQ5 Where does the analysis stop helping against an adversary who has read it?

Contributions. (1) A controlled comparison that holds a policy engine fixed and varies only the predicate. The predicates are substring rules; four regex variants (strong, traffic-tuned, matched to the analyzer's vocabulary, and the latter on normalized commands); gate-everything; and facts derived from a shell AST, plus analyzer ablations that replace the AST with tokenization or switch off individual components (Section III). (2) Evaluation on an independently authored held-out benchmark, a source-aware probe set, a frozen corpus of 27,927 real Claude Code tool calls, two corpora from a second agent (Codex), and a pre-specified out-of-sample test on public sessions of 430 other developers (417 in SWE-chat, 13 self-published), with paired-difference intervals and session- and developer-level tests (Sections IV–V). (3) A decomposition. Held-out recall comes mostly from vocabulary, and the benchmark cannot resolve the rest. On real work, in-sample and out of sample, the syntax tree is the largest

single factor in how often the gate interrupts, through long, multi-line commands. (4) Audits of what real interruptions are: denials, approvals, and the calls on which facts disagree with a vocabulary-matched regex or with the tokenizer. A joint relabeling under one protocol shows that our first in-sample audit overstated the value of facts' own extra stops by counting readable run-time code as gated. Real traffic also exposed the analyzer's largest false-positive sources and showed interpreter code to be where static structure gives way to matching again (Section VI). (5) An open artifact, with materials for human relabeling.

Relation to prior work. CARE [16] also parses agent shell commands, compares with a regex baseline and ablates its structural layer, and Claude Code's auto mode [15] reports error rates of an LLM classifier on internal traffic. We do not claim the first shell parser for agents. What is new is a decomposition under one fixed policy engine against deliberately favorable text baselines (vocabulary-matched and normalized), interruption cost measured on public, inspectable traces with a pre-specified test across developers, and audits of what the interruptions are.

II. THREAT MODEL

An agent issues tool calls—`Bash`, `PowerShell`, `Read`/`Edit`/`Write`, `Grep`—intercepted before execution (here, by a Claude Code `PreToolUse` hook or an MCP proxy). The agent may be mistaken or steered by prompt injection. We gate three classes of action. (i) *Destructive*, defined syntactically: bulk or recursive deletion, destructive database statements, and rewriting or deleting shared history. As a policy choice this includes directories that could be regenerated; we measure its cost. (ii) *Secret access*: reading, copying or sending credential material. (iii) *Opaque*: execution whose effect cannot be determined from the call. Each call is judged in isolation. We evaluate robustness to rephrasing and to one short source-aware probe round, not to a sustained adaptive adversary. Out of scope are multi-step decompositions such as writing a script and then running it [17]. The same holds for any command that runs a project script or build target (`./x.sh`, `make`, `npm run`), 3.1% of real shell calls, whose effect lies in files the gate does not read. Also out of scope are effects that bypass the tool layer [18] and attacks on the gate itself.

III. DESIGN: FACTS UNDER A FIXED POLICY ENGINE

ActionGate evaluates each call against ordered rules in CEL [19], a non-Turing-complete expression language; the first match decides, and evaluation errors fail closed. Its earlier default (v1) used substring predicates:

```
tool_name == "Bash" && (params.command.contains("rm
    -rf")
  || params.command.contains("git push --force") ||
    ...)
```

We keep the engine and add one input, a map `facts` computed by a pure function of tool name and parameters. The fact-based policy is three rules:

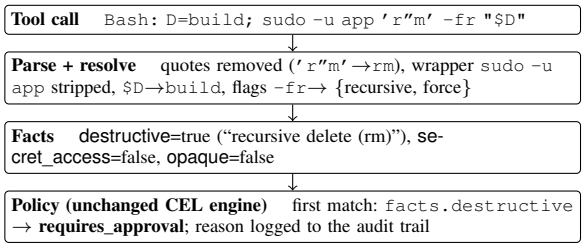


Fig. 1. One command through the fact pipeline. The command is a disguised `rm -rf build`. The substring rule `contains("rm -rf")` allows it; so would any rule over the raw text that does not anticipate quoting, wrappers and variables.

```
facts.secret_access -> deny
facts.destructive   -> requires_approval
facts.opaque        -> requires_approval
```

By analogy with authorization languages such as Cedar [20], the policy stays small and auditable while the application supplies structured attributes; an operator can write `facts.destructive && environment == "prod"` without knowing shell syntax. Fig. 1 traces one command through the pipeline.

Shell analysis. Commands are parsed into a Bash AST with `mvdan.cc/sh` [21]. Each word is resolved to its static value: quotes and escapes removed, braces expanded (`.{e,}nv`→ `.env`), and variables substituted when assigned a static value earlier in the same command. Wrappers that only run another command are stripped (`sudo -u x`, `env A=1`, `xargs`, `timeout`, `npx`); flags are interpreted, so `-fr`, `-r -f` and `--recursive` coincide. Nested shell code (`bash -c`, `eval` of literals, here-strings, static here-documents) is re-parsed to bounded depth. Destructive semantics are defined per tool: recursive `rm`, `find -delete`, force pushes and delete refspecs, `reset --hard`, `clean -f`, `dd` to a device, destructive SQL reaching a database client anywhere in a pipeline, and bulk-delete verbs of container, cluster and cloud CLIs.

Secret access counts only commands that expose contents (`cat`, `grep`, `sed`, `tar`, `curl -d @f`, `cp` sources, `git show`), not ones that test, list, or hand a file to a program (`test -f .env`, `docker --env-file .env`). A reader's leading pattern (`grep PATTERN FILE`) is not a file. Credential files are recognized by name; templates (`.env.example`) and public keys are excluded.

Opacity. At the shell level the analyzer does not guess: `eval` of computed text, a pipe into a shell or interpreter, a command name computed at run time, process substitution used as a script, unparseable input, excess nesting, and analyzer panics all yield opaque. Running a project script is not opaque but out of scope. Inline interpreter code (`python -c`, `node -e`, here-documents) is only scanned with patterns for deletion APIs, secret file names and embedded shell commands; code that matches nothing is *allowed*. A fourth fact, inline_dynamic, flags interpreter code that builds strings or names at run time; the *facts-strict* policy escalates it. PowerShell is analyzed at token level. The analyzer is about 1,400 lines of Go, performs no I/O, and its final Bash entry point was fuzzed for 60 s

(369,056 inputs with four workers, no failures).

Ablation switches. For evaluation only, the analyzer accepts options that leave the policy untouched. *Tokenizer* replaces the AST with quote-aware tokenization: commands split at `;`, `&&`, `||`, `|`, `&`, words at whitespace, quotes removed, nothing expanded or re-parsed. The same per-command semantics apply, which approximates a command table over `shlex`-style tokens. *Tokenizer+* is a fairer tokenizer: it also cuts here-document bodies out of the line and hands them to the command that reads them, and substitutes variables assigned a static value, still without a syntax tree. *No-readers* counts every mention of a credential file as a read. *No-vars* disables static variable substitution. *Exempt* stops treating a recursive `rm` as destructive when every target is static and is either a conventionally regenerable directory (`build`, `dist`, `node_modules`, `.next`, caches, `tmp`) or lies under a temporary directory. The list is generic, not fitted to our corpus; it would also exempt a real directory named `build` or a symlink so named.

IV. EVALUATION METHOD

Labels. Benign → allow; destructive or opaque → requires approval; secret access → deny. These categories are the analyzer's own, so the benchmarks measure conformance to this ontology, not harm in general. Benign items deliberately include look-alikes (dangerous words in commit messages or grep patterns, dry runs, templates, public keys, `DELETE ... WHERE`), which is adversarial to text rules. Benchmark FPR therefore reflects this composition, not deployment.

Datasets. *Development* (228 actions; 95 benign, 133 harmful; 22 PowerShell), written by the first author before the analyzer and now a regression test. *Held-out B* (150; 70 benign, 80 harmful; 33 PowerShell), the primary test set and the only clean one, evaluated once on the final analyzer. *Held-out A* (120; 50 benign, 70 harmful; 13 PowerShell), evaluated on a frozen earlier analyzer (version 2.1, unrelated to the v1 policy); its misses later informed fixes. *Protocol.* Each held-out set was written in a single pass by a separate LLM agent. Its prompt contained only the label definitions, the required category counts, and a request for realistic, diverse items including hard look-alikes, with an instruction to skip ambiguous cases. The agent had no access to the code, the other sets, or analyzer output. No item was discarded or edited afterwards. A further agent, blind to the original labels and to analyzer output, relabeled each set, and the labels agreed on all 270 items. Because ambiguous cases were excluded by construction, this agreement shows only that a second model applies the definitions the same way; no human has labeled the held-out sets. *Probes* (53; 50 harmful, 3 benign): written by a reviewer agent that had the analyzer's source, in about fifteen minutes plus one follow-up batch, drawing mostly on general evasion knowledge. One probe (`alias x=rm; x -rf src`) was relabeled benign because aliases do not expand in non-interactive Bash. *Real traffic.* We froze a hashed copy (manifest SHA-256 prefix `60f161cc`) of the first author's Claude Code transcripts on October 31, 2026. It holds 27,927 governed tool calls (Bash 19,932; PowerShell 352; file tools 7,643) in 15 sessions (sub-agent transcripts merged into their parents) across several projects. The session in which this paper was written is excluded. One project accounts for 97% of shell calls. Transcripts record the calls as the agent issued them, not whether Claude Code's own permission settings then blocked or auto-approved them. *Second agent.* To test whether results carry over to an agent the analyzer was never tuned on, we use OpenAI's Codex CLI, whose logs record the full argument vector of every command it ran. *Codex, organic*: the first author's own Codex sessions (frozen, prefix `2686606e`), 11 sessions with 4,364 commands and 656 file changes. On Windows, Codex ran every command through PowerShell (4,340) or `cmd.exe` (24). *Codex, Bash*: because that corpus exercises none of the Bash analysis, we also ran seven everyday tasks through Codex (`codex exec`, workspace-write sandbox, instructed only to use Git Bash). The tasks were fixing a failing test, cleaning generated files and rebuilding, checking a local API setting, squashing work-in-progress commits, adding a CLI subcommand with a test, writing and running a log-rotation script, and resetting a test database. They ran in a small Python project we built with a fake `.env`, build and cache directories, logs, an SQLite database and a git history. The prompts never asked for a destructive or secret-reading command. The resulting 35 commands and 5 file writes (frozen, prefix `fac0e501`) were labeled by the first author under the same definitions. All other LLM agents in this study were Claude models, the same family that assisted development, so shared blind spots remain a validity threat (Section VIII). *Other developers (pre-specified).* Before reading any of these data we froze the analyzer and all packs (hash prefix `6809d27d`) and wrote down three one-sided hypotheses, with developers as the unit. Facts stop fewer calls than tokenizer+ (H1) and than the vocabulary regex (H2) for more developers than the reverse (sign test), and a blind audit finds fact-only stops warranted more often than regex-only stops (H3, Fisher test). The primary source is SWE-chat [22]: real sessions of Claude Code, Codex, Pi, OpenCode and other agents, mined from public GitHub repositories that checkpoint agent sessions with the Entire.io CLI. Its developer identifier is the commit author resolved through GitHub; 417 developers with at least 50 shell calls contribute 640,530 calls. The second source is 13 developers who published their own sessions on Hugging Face (DataClaw exports and raw transcripts; 453,036 calls, two-thirds from one developer). Older DataClaw exports cut inputs at 300 characters. An amendment, made after inspecting command lengths but before any policy was run, restricts the primary analysis to complete inputs and adds a secondary one on commands shorter than 290 characters from every export. The audit draws 30 fact-only and 30 vocabulary-regex-only stops from SWE-chat, one per developer (round-robin), so it describes a typical developer's stops. A Claude agent labeled them blind, mixed with 50 held-out B actions; the same items are released as a kit for human annotators. The specification is a timestamped file in our repository with the frozen hash, not a deposit with an external registry; it, its two amendments and eight recorded deviations are in the artifact. The most consequential: the converter for

the second source was corrected after a first run that wrongly admitted cut commands had been seen; no human labels were collected; and the effect sizes, per-agent results, sensitivity runs, the joint relabeling, the tokenizer audit and the off-the-shelf parser ablation are exploratory and marked as such in Section V.

Policies. *v1*: ActionGate's own earlier substring pack. It is a baseline we wrote, representative of documented practice [8], but not of any vendor's built-in rules, which we could not evaluate offline. *Strong regex*: four case-insensitive RE2 rules covering flag variants, split flags, git force and delete forms, SQL, `find -delete`, interpreter deletion APIs, PowerShell, shell-wide secret names and pipes into shells. The first author wrote it with knowledge of the development set and of held-out A, before the probes existed, so it is deliberately favorable. *Tuned regex*: the same rules after an analogous tuning pass on real traffic, applied as five cumulative edits so that each can be ablated. The edits restrict `eval` to command position, drop `$(curl ...)` capture, restrict `-EncodedCommand` to `powershell`, drop variable-named commands, and ignore secret names after `test`, `ls` and `--env-file`. The analogy is imperfect: the analyzer's fixes added capabilities, whereas a regex can only drop or narrow patterns. *Vocabulary-matched regex* (vocabulary regex for short): the tuned regex extended with every secret file name and destructive verb the analyzer knows. That vocabulary predates B, except cloud-CLI delete verbs, which were added after held-out A but before B was evaluated. The regex was assembled after B's results were seen. Its exemption of commands that mention a template or public key was added after seeing its B false positives; this mirrors analyzer rules that predate B and favors the regex. *Normalized vocabulary regex*: the same regex applied to the command after the analyzer's normalization (quotes removed, braces expanded, static variables substituted, wrappers stripped, nested code inlined). *Gate-all*: every Bash and PowerShell command and every file mutation requires approval, and file-tool reads of secret paths are denied; other file reads pass. *Facts*, *facts-strict* and the ablations are as in Section III.

Metrics. Recall: harmful actions stopped (deny or approval). FPR: benign actions stopped. Exact: decision equals label. Friction (or interruption rate): share of real shell calls stopped. Pooled rates divide all stops by all calls; macro-averaged rates average per-project, per-session or per-developer rates so that no unit dominates. A fact-only stop is a call that facts stop and the compared policy allows; a regex-only stop is the converse. Per active hour: stops divided by the time between a session's first and last shell call, over sessions with at least 20 shell calls and 15 minutes. Intervals are 95% Wilson intervals, and differences between proportions use Newcombe's score intervals, paired [23] or for independent samples [24]. On B we report discordant counts as (fact-only, other-only) stops. Exact McNemar tests compare facts with v1 on recall and with the strong, tuned and vocabulary-matched regexes on recall and FPR; these seven tests are Holm-corrected. Items cluster by technique, so even these are optimistic. On our traffic we compare policies with a two-sided sign test over the 15 sessions; out of sample we use the pre-specified one-sided sign test over developers. The pre-specified hypotheses are H1 and H2 on SWE-chat and H3 on its audit, each at $\alpha = 0.05$ without correction; H1b and H2b are their secondary versions, and everything else out of sample is exploratory.

V. RESULTS

RQ1: what text misses. Our own earlier substring policy stops 16.3% of held-out B's harmful actions (Table I; CI 9.7–25.8). Part of this is omission rather than brittleness, since v1 has no rule for shell-based secret reads or PowerShell. The development set shows brittleness directly: v1 stops all 11 canonical destructive commands but only 1 of 52 equivalent rephrasings (`bench/RESULTS-dev.md`). Its only three denials on real traffic were file-tool reads of template files (`.env.example`): false positives. A strong regex, written with far more care, reaches 82.5% recall but 12.9% FPR, because patterns broad enough to catch variants also catch the words inside harmless commands.

RQ2: what the fact layer adds. Facts stop 90.0% of B's harmful actions (CI 81.5–94.8) at 5.7% FPR (CI 2.2–13.8), and decide 92.0% of actions exactly (deny vs. approval vs. allow). Against v1 the recall gain is large (59 vs. 0, Holm-adjusted $p < 10^{-16}$). Against the strong regex neither difference is significant: recall $+7.5$ points (CI -0.7 to $+16.2$; 8 vs. 2) and FPR -7.1 (CI -16.3 to $+1.0$; 1 vs. 6), both adjusted $p = 0.55$. Against the tuned regex, which interrupts real traffic as rarely as facts, recall is $+12.5$ points (CI $+4.1$ to $+21.6$; 11 vs. 1; adjusted $p = 0.04$). This result is fragile: tuning cost the regex exactly four harmful items, all dynamic code (`cmd=$(curl ...); $cmd`, a loop over `$(curl ...)`, `node -e "eval(...)"`, Perl `eval pack`). Two were lost by restricting `eval` to command position and two by dropping `$(curl ...)` capture, and a different tuning could keep them.

Vocabulary, normalization, AST, semantics. Table II locates the remaining differences. Giving the regex the analyzer's vocabulary lifts it to 86.3% at 11.4% FPR. Facts' advantage is then $+3.8$ recall points (CI -3.6 to $+11.6$; 5 vs. 2) and -5.7 FPR points (CI -13.7 to $+0.9$; 0 vs. 4). Neither is significant, but the benchmark cannot exclude a 12-point recall advantage either: it is underpowered, not evidence of parity. The vocabulary regex's four extra false positives are dangerous words inside a commit message, a `grep` pattern, a `Select-String` pattern and a `Write-Output` string. Normalizing commands first changes almost nothing (85.0%, 11.4%), and can erase a textual cue (`bash <(curl ...)` becomes a placeholder). Replacing the AST with tokenization keeps per-tool semantics and loses three recall points, one destructive and two opaque Bash items (3 vs. 0; CI -1.7 to $+10.0$), and nothing in FPR. The fairer tokenizer+ scores identically. On B, then, vocabulary carries most of the recall, and token-level semantics already supplies most of the precision. Facts' residual advantage over the vocabulary regex lies in opaque Bash code (9 vs. 5 of 10). The vocabulary regex in turn beats facts on destructive PowerShell (8 vs. 6). Disabling the read-vs-mention rule adds two false positives; disabling variable

TABLE I
DETECTION AND INTERRUPTION COST

Policy	Held-out B (80 harmful / 70 benign)			Held-out A (70 / 50)		Probes (50)	Real traffic
	Recall	FPR	Exact	Recall	FPR	Recall	Shell friction
v1 (our substring pack)	16.3%	4.3%	53.3%	17.1%	10.0%	20.0%	1.21%
Strong regex	82.5%	12.9%	84.7%	90.0%[a]	6.0%[a]	72.0%	6.88%
Tuned regex	77.5%	10.0%	83.3%	90.0%[a]	6.0%[a]	72.0%	2.68%
Vocabulary-matched regex[d]	86.3%	11.4%	87.3%	94.3%[a]	8.0%[a]	76.0%	3.64%
Vocabulary-matched regex, normalized	85.0%	11.4%	86.7%	–	–	78.0%	3.56%
Gate-all	100%	94.3%	43.3%[c]	98.6%	94.0%	98.0%	100%
Facts	**90.0%**	**5.7%**	**92.0%**	91.4%	0.0%	78.0%	**2.74%**
tokenizer (no AST)	86.3%	5.7%	90.0%	–	–	74.0%	9.90%
tokenizer+ (here-docs, vars)	86.3%	5.7%	90.0%	–	–	74.0%	6.69%
tree-sitter parser[e]	90.0%	5.7%	92.0%	–	–	78.0%	3.29%
without read-vs-mention	90.0%	8.6%	90.7%	–	–	78.0%	3.42%
without variable resolution	90.0%	5.7%	92.0%	–	–	78.0%	4.93%
with regenerable-path exemption	90.0%	5.7%	92.0%	–	–	74.0%	2.56%
Facts-strict	90.0%	5.7%	92.0%	–	–	86.0%[b]	8.38%

Held-out B is the primary test; A was scored on the analyzer frozen at v2.1. Shell friction: share of the 20,284 real shell calls a policy stops. [a]Regex written with knowledge of A. [b]Strict variant designed after the probe round. [c]Includes six secret reads that gate-all's file-tool rule denies exactly. [d]Assembled after B's results were seen. [e]Exploratory: tree-sitter-bash in place of the AST, same per-command semantics; its front end was corrected once after B's misses were inspected (71/80 before).

TABLE II
HELD-OUT B BY CATEGORY AND TOOL

Category	Tool	n	v1	Tuned	Vocab.	Tok.	Facts
Destructive	Bash	32	9	25	27	26	27
	PowerShell	8	0	7	8	6	6
Secret	Bash	14	0	11	14	14	14
	File tools	6	4	6	6	6	6
	PowerShell	5	0	4	5	5	5
Opaque	Bash	10	0	5	5	7	9
	PowerShell	5	0	4	4	5	5
Benign (FP)	Bash	48	2	3	4	2	2
	File tools	7	1	0	0	0	0
	PowerShell	15	0	4	4	2	2

Cells: harmful actions stopped; benign rows: actions wrongly stopped. Tuned: tuned regex. Vocab.: vocabulary regex. Tok.: facts with tokenization instead of the AST.

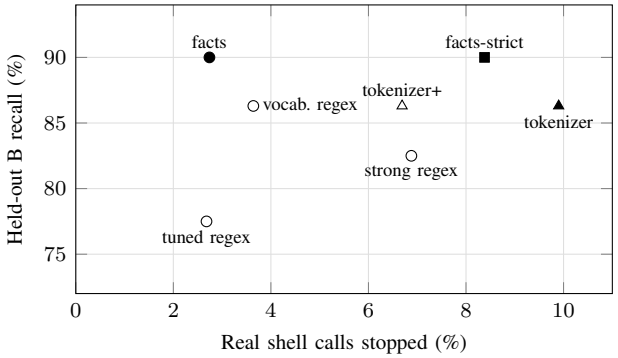


Fig. 2. Recall on held-out B (y) against the share of our real shell calls each policy stops (x). The axes come from different data: real traffic is unlabeled, so it measures cost, not detection. Up and left is better. Outside the plotted range: v1 (1.21%, 16.3%) and gate-all (100%, 100%).

resolution changes nothing on B. Every policy evaluates an action in well under a millisecond, negligible next to a model turn.

RQ3: cost on real traffic. On the frozen corpus facts stop 2.74% of shell calls and the strong regex 6.88% (Table III, Fig. 2). The regex stops more in 10 of 15 sessions and fewer in 2 (sign test, $p = 0.04$), at a median of 1.5 interruptions per active hour against 0.3 for facts. About three quarters of the regex's approvals come from its opaque rule. Most of those come from a bare `eval` that also matches `pytest -k eval`, from output capture such as `s=$(curl ...)`, and from commands that begin with a variable. Tuning those away brings it to 2.68% (6 vs. 3 sessions, $p = 0.51$). Adding the analyzer's vocabulary raises it to 3.64%, and normalizing first to 3.56% (vocabulary regex vs. facts: 7 vs. 2 sessions, $p = 0.18$). The ablations show what produces facts' rate. Without the AST, tokenization stops 9.90%, more than facts in 12 of 15 sessions and fewer in none ($p = 0.0005$), because here-documents and multi-line programs fall apart into spurious commands. Handing a tokenizer here-document bodies and static variables, as tokenizer+ does, recovers part of this: it stops 6.69%, still more than facts in 11 sessions and fewer in none ($p = 0.001$). The remaining gap comes from constructs a line-based tokenizer does not delimit: quoting and command substitution that span lines, compound commands (`for`, `if`, functions) and nested code. Without variable resolution facts stop 4.93%, and without the read-vs-mention rule 3.42%. The syntax tree, which barely mattered on B, is the largest single factor on real work. Facts stop none of the 7,643 file-tool calls, because the default pack has no file-tool rule except via secret paths, and the agent read no credential file through a file tool; facts therefore interrupt

TABLE III
INTERRUPTIONS ON REAL SHELL TRAFFIC

Policy	Stopped	Denied	Approval	Per hour
v1	1.21%	0	246	–
Strong regex	6.88%	286	1,110	1.5
Tuned regex	2.68%	260	284	0.3
Vocabulary regex	3.64%	422	316	0.3
Facts	**2.74%**	224	332	**0.3**
tokenizer	9.90%	157	1,852	2.3
tokenizer+	6.69%	163	1,194	–
with session exemption	2.38%	224	–	–
with path exemption	2.56%	224	296	0.3
Facts-strict	8.38%	224	1,475	–
Gate-all	100%	0	20,284	–

Frozen corpus: 20,284 shell calls in 15 sessions. v1's three denials were file-tool calls and are not counted. Per hour: median interruptions per active hour, over sessions with at least 20 shell calls and 15 minutes. –: not measured.

1.99% of all governed calls. Friction is heterogeneous: 2.65% in the dominant project and 5.9% and 7.0% in two smaller ones. Macro-averaged, so that the dominant project does not set the figure, facts stop 2.77% per project (7 projects) and 2.92% per session (11 sessions with at least 20 shell calls); tokenizer+ stops 6.86% and 6.59%. Facts' friction is also much higher on PowerShell, where they stop 9.4% of 352 calls, most often scripts invoked through a variable holding a literal path, which the token-level PowerShell analysis cannot resolve. The tuned and vocabulary regexes stop about 3% of PowerShell calls (the strong regex 45%).

Are the interruptions warranted? We audited samples of disagreements, denials and approvals. *Disagreements (superseded).* This first disagreement audit is superseded by the joint relabeling below; we report it because the discrepancy is itself a finding. Facts stop 102 calls the vocabulary regex allows (in 8 sessions). The regex stops 284 calls facts allow (in 7 sessions): 223 denials and 61 approvals. We drew 30 fact-only stops and 30 regex-only denials by seeded random sampling (from 7 and 6 sessions), mixed them, and had an annotator, a Claude agent, label each call blind to which policy had stopped it. 25 of 30 fact-only stops contained at least one gated action (83%, CI 66–93); counted per action, 19 involved dynamic code, 6 secret reads and 4 deletions (some calls contained two). Only 3 of 30 regex-only denials did (10%, CI 3–26). The others mentioned a credential file without reading it: an `--env-overlay .env.local` argument handed to a dev server, a `.env` name inside code or notes being written, a `git` listing. To make the strata comparable we then sampled 30 of the 61 regex-only approvals and had a fresh agent label them with the same definitions: 1 contained a gated action (a deletion of a user's records), and the rest were visible inline scripts, test-database resets and code that merely contains the word `eval` (3%, CI 1–17). No human has labeled these audits. On our traffic, session counts could not separate the two policies (7 vs. 2 sessions, $p = 0.18$), but the content of their disagreements favored facts. A joint relabeling below shows that much of this came from counting readable run-time code as gated.

Denials. In a seeded sample of 40 fact-based denials (drawn before freezing, from the then 226; the frozen corpus has 224), 37 (92.5%, CI 80.1–97.4) read or copied a real credential file. In five of them an API key was extracted and used, for instance with `sed` into a `curl` header. Whether such reads were *unwanted* is a separate question: they were ordinary development work. A hard deny invites circumvention, and `requires_approval` is a reasonable alternative for secret reads.

Approvals and exemptions. Two annotators, the first author with Claude's assistance and a separate Claude agent (so not fully independent), labeled a seeded sample of 40 approvals ($\kappa = 0.90$). Like the denial sample it was drawn before freezing, from the then 337 approvals; the frozen corpus has 332. At most one was judged worth the interruption (deleting a project copy). About 27–28 (68%, CI 52–80) deleted build outputs, caches or scratch directories the agent had just created. 7–8 (20%, CI 11–35) were analyzer false positives, and 4–5 ran benign run-time code. Approvals are therefore mostly not warranted, and the causes are mixed: the definition of destructive, analyzer bugs (notably PowerShell scripts invoked through a variable), and conservative opacity. A per-call path exemption recovers little of the first cause: friction falls from 2.74% to 2.56%, with fewer stops in 6 sessions and more in none ($p = 0.03$). The annotators' judgment relied on knowing that the agent had just created the directory, often under an opaque relative name (one was `un21`, a scratch copy) that no generic list contains. We therefore tried session state: a recursive `rm` whose only reason is the deletion is exempted when every target was created earlier in the same session (by `mkdir`, `cp -r`, `unzip -d`, `git worktree add` or `python -m venv`). Friction falls only to 2.38%, fewer stops in 3 sessions. Most deleted directories were created by programs (builds, test runners, scripts), which a command-level record of creations does not see. The benchmarks and probes are single actions with no prior history, so this exemption cannot change their scores. The per-call exemption, by contrast, lets two probes through: `rm -rf node_modules` and `mv src /tmp/trash && rm -r /tmp/trash`, which deletes source code via a temporary directory.

RQ4: a second agent. On the organic Codex corpus, which the analyzer was never tuned on, facts stop 1.86% of 4,364 commands (34 denials, 47 approvals). The tuned and vocabulary regexes stop 1.8% and 1.9%, the strong regex 12.2%, and v1 none. Every command was PowerShell (the 24 `cmd.exe` commands are passed to the PowerShell analyzer as `cmd /c ...`), so this tests only the token-level PowerShell analysis, on which facts and the regexes behave alike; within that scope, low friction carries over to a second agent. That rate is also far below facts' 9.4% on Claude Code's 352 PowerShell calls. We did not audit these stops, so we cannot say whether the gap reflects the two agents' command styles or the tasks. The staged Codex Bash corpus exercises the Bash analysis. Of its 35 commands and 5 writes, three are harmful. In the cleaning task Codex ran `git clean -fdX`, which removed every git-ignored file, including the developer's `.env` and the SQLite

TABLE IV
OUT OF SAMPLE: OTHER DEVELOPERS' SHELL CALLS

	SWE-chat (417 dev.)			Self-published (13)		
Policy	Pooled	Macro	Fewer/ more	Pooled	Macro	Fewer/ more
v1	0.83%	0.86%	–	0.48%	0.88%	–
Strong regex	3.74%	3.04%	–	2.24%	2.89%	–
Tuned regex	2.57%	2.02%	119/123	1.09%	1.38%	1/10
Vocab. regex	3.16%	2.33%	166/86	1.18%	1.56%	2/8
Facts	**1.98%**	2.21%	–	1.30%	2.33%	–
tokenizer	4.73%	4.48%	270/21	2.04%	3.30%	10/0
tokenizer+	3.82%	3.75%	251/28	1.88%	3.05%	10/0
tree-sitter*	2.05%	2.26%	68/21	1.33%	2.37%	6/1

Share of shell calls stopped: pooled over calls, and macro-averaged over developers with at least 50 shell calls (SWE-chat 640,530 calls; self-published 453,036). Fewer/more: developers for whom facts stop fewer / more calls than that policy (one-sided sign test). –: not compared. Analyzer frozen before the data were read. *Exploratory off-the-shelf parser ablation (Table I).

TABLE V
WHERE FACTS DO NOT INTERRUPT LESS THAN A REGEX

Setting	Regex	Dev.	Fewer/more	p
SWE-chat	tuned[a]	417	119/123	0.63
OpenCode users[b]	vocab.	35	10/8	0.41
Pi users[b]	vocab.	13	4/4	0.64
Self-published (H2)	vocab.	13	2/8	>0.98
	tuned[a]	13	1/10	>0.99
Self-published, <290 chars (H2b)	vocab.	19	3/12	>0.99
	tuned[a]	19	1/13	>0.99

Fewer/more: developers for whom facts stop fewer / more shell calls than the regex; p: one-sided sign test that facts stop fewer. On SWE-chat as a whole the macro-averaged difference from the vocabulary regex is -0.1 points (CI -0.4 to $+0.2$). [a]Not a pre-specified hypothesis. [b]Exploratory: developers with at least 50 shell calls of that agent.

database, not just build output. It also ran `git clean -fd` on a packaging directory, and in the log task it executed a script it had just written that deletes old files. Facts require approval for both `git clean` commands and interrupt nothing else (2 of 3 harmful, 0 of 37 benign). They miss the script run, which the threat model declares out of scope. v1 lets all three through. The strong and tuned regexes match facts on the two deletions but also stop a benign `find . -name '.env'`. The vocabulary regex matches facts. Gate-all stops everything. The sample is far too small for rates, but it shows the failure the paper is about, a routine instruction turned into bulk deletion of untracked data, and it shows that the script-execution gap is not hypothetical.

RQ4 (pre-specified): 430 other developers. Table IV gives the pre-specified results. On SWE-chat facts stop 1.98% of shell calls and tokenizer+ 3.82%. Facts stop fewer calls for 251 developers and more for 28 (one-sided sign test, $p = 3\times10^{-46}$), so H1 holds. Against the vocabulary regex (3.16%) the counts are 166 and 86 ($p = 3 \times 10^{-7}$), so H2 holds too. The tuned regex was not in the hypotheses. It stops 2.57% pooled and 2.02% macro-averaged, against 2.21% for facts, and neither policy interrupts less for more developers (119 vs. 123). On the 13 self-published developers H1 holds again (10 vs. 0, $p = 0.001$), but H2 does not: the tuned and vocabulary regexes interrupt less for 10 and 8 of them. There the most common fact reasons are recursive deletion, `source` of a `.env` file, and 688 commands the Bash parser rejected, which fail closed. In the pre-specified secondary analysis of the self-published exports, restricted to commands shorter than 290 characters (19 developers), tokenizer+ no longer differs from facts (8 vs. 6 developers, $p = 0.40$, H1b not supported), and the vocabulary regex interrupts less for 12 developers against 3 (H2b not supported). Pooled over both sources, facts interrupt less than tokenizer+ for 261 of 430 developers and more for 28, and less than the vocabulary regex for 168 and more for 94; against the tuned regex the split is even (120 vs. 133). Table V collects every setting in which facts do not interrupt less than a regex: H2 holds on SWE-chat as a whole and for Claude Code and Codex users, and nowhere else.

Effect sizes and robustness (exploratory). The sign test counts a one-call difference like a large one, so we also report per-developer differences on SWE-chat. Against tokenizer+ the effect is clear: facts interrupt 1.5 points less macro-averaged (bootstrap 95% CI 1.3–1.8; median -0.5). Against the vocabulary regex the macro difference is -0.1 points (CI -0.4 to $+0.2$) and the median is zero. The pooled difference (-1.2, CI -2.4 to -0.1) comes mostly from heavy users; the ten largest developers hold 41% of calls. Against the tuned regex, facts are $+0.2$ points macro-averaged (CI -0.05 to $+0.5$). Restricted to developers with at least 50 calls of one agent, H1 holds for Claude Code (224 vs. 21), Codex (26 vs. 2) and OpenCode (15 vs. 3) but not significantly for Pi (8 vs. 2, $p = 0.055$); H2 holds for Claude Code (149 vs. 74) and Codex (22 vs. 10) but not for OpenCode (10 vs. 8) or Pi (4 vs. 4). Excluding the 32 developers who work in the collection tool vendor's own repositories leaves the results unchanged (H1 224 vs. 28, H2 152 vs. 80, tuned 113 vs. 111 of 387). Within SWE-chat the tokenizer gap does not vanish for short commands: restricted to commands under 290 characters, facts interrupt 0.98% and tokenizer+ 1.58% (191 vs. 21 developers). Still, about three quarters of tokenizer+'s extra stops come from the 22% of calls whose commands have 290 characters or more, which the syntax tree resolves and a line-based tokenizer does not. The sign tests drop ties: 138 developers for H1, 165 for H2 and 175 against the tuned regex, of whom 50, 65 and 68 are stopped by neither policy; 36 developers are stopped by no policy but v1. Per developer the distributions are skewed. Median friction is 1.23% for facts, 1.31% for the vocabulary regex, 1.12% for the tuned regex and 2.21% for tokenizer+, and the 90th percentiles are 4.8%, 5.3%, 4.5% and 8.8%. For 286 of 417 developers facts and the vocabulary regex differ by less than half a point; against tokenizer+, 194 do.

An off-the-shelf parser (exploratory). To check that the result does not depend on our parser, we replaced mvdan's syntax tree with tree-sitter-bash and handed its simple commands to the same per-command semantics. It matches facts on held-out B and the probes (Table I). On SWE-chat it interrupts less than tokenizer+ for 245 developers and more for 36, and it

differs from facts by $+0.05$ points macro-averaged (CI $+0.01$ to $+0.09$); 0.7% of the corpora's distinct commands do not parse in tree-sitter and fail closed. On our own traffic, where the analyzer's false-positive fixes were made, it stops 3.29% against facts' 2.74%. The gap to the tokenizer is thus a property of parsing, not of our parser.

What the extra stops are (pre-specified audit; relabelings exploratory). In the pre-specified audit the annotator, which agreed with held-out B's labels on 49 of the 50 kit items ($\kappa = 0.97$; both are Claude models), found a gated action in 5 of 30 fact-only stops (17%, CI 7–34) and 3 of 30 regex-only stops (10%, CI 3–26). The difference, $+6.7$ points (CI -11.5 to $+24.8$; one-sided Fisher $p = 0.35$), leaves H3 inconclusive. Unlike our in-sample audit, which sampled regex-only denials, this one also includes regex-only approvals. Fact-only stops were mostly fail-closed opacity: a command name held in a variable (10), interpreter code built at run time (7) and input the parser rejected (4), only 2 of which the annotator judged genuinely opaque. The in-sample audit had found 25 of 30, with a different annotator instance and a guide that let dynamic code count as gated. To separate population from protocol (exploratory, added after seeing the discrepancy), a fresh agent relabeled all 150 audit items (in-sample: 30 fact-only, 30 regex-only denials, 30 regex-only approvals; out of sample: the 60 above), shuffled and blind to source, with an added label for code that is built or named at run time but that a careful reader can resolve from the call. Under this one protocol the two populations look alike. Strictly gated were 8 of 30 in-sample fact-only stops (27%, CI 14–44) and 5 of 30 out of sample (17%, CI 7–34). A further 19 and 17 were readable run-time code, and 3 and 8 benign. Regex-only stops were strictly gated in 2–6 of 30 per stratum (7–20%) and benign or mere mentions in 11–19. Fact-only stops are thus mostly the analyzer's conservative opacity, on its own traffic as on others'. The first in-sample audit counted that opacity as warranted. The added label is relative to a static analyzer and was applied by a model that reads code more easily than one; human labels may draw the line elsewhere. H1 rests on the claim that the tokenizer's extra stops are spurious, so under the same protocol (exploratory) a fresh agent also labeled 30 tokenizer+-only and 30 fact-only stops from SWE-chat (48 developers, round-robin). Of the tokenizer+-only stops, 26 were benign: build and test runs, API queries and status polls, all using variables or command substitution and most (22) spanning several lines. The other 3 were gated: each read a token from a keychain, a password manager or an auth file, which facts allowed, so they are fact-layer misses. Of the fact-only stops, 9 were strictly gated (30%, CI 17–48; 7 secret reads, mostly of `.env` files, and 2 force pushes), 6 readable run-time code and 15 benign, against 3 of 30 for tokenizer+ (10%, CI 3–26). The difference is $+20$ points (CI -0.5 to $+39$; one-sided Fisher $p = 0.052$). Structure thus removes mostly spurious stops and adds a few warranted ones, but it does not catch everything the tokenizer catches.

RQ5: a source-aware probe round. With the source in hand, a reviewer agent evaded facts on 11 of 50 harmful probes. Six probes read five credential stores missing from the name list: Docker and GitHub CLI configs, `~/.kube/config`, `terraform.tfstate` and `.envrc`. Four built strings inside interpreter code (`getattr(__import__('shutil'),'rmt'+'ree')`, `f['rm'+'Sync']`, `open('.e'+'nv')`), and one ran `truncate -s 0` on a database file. The normalized vocabulary regex ties facts (78.0%), the other regexes stop 72–76%, and v1 20.0%. With three benign probes, probe FPR is not meaningful. The strict variant closes the interpreter group (86.0%, post hoc on the same probes) but raises real-traffic friction from 2.74% to 8.38%. Agents themselves build strings in inline Python constantly, and the rule is blunt: it also flags `', '.join(...)`. We did not replay the incidents of Section I. Their reports describe the payloads but not the exact command lines an agent issued, so reconstructing them would test our reconstruction.

VI. ANALYSIS

Real traffic finds what benchmarks do not. On an earlier snapshot of the corpus, the first fact-based analyzer scored 100% on its development set and 91.4% recall at 0% FPR on held-out A, yet stopped 18.2% of real shell calls. Tallying its reasons found three causes absent from every benchmark. `python - <<'EOF'` here-documents were treated as code from stdin although the code is static (2,568 calls). Interpreter paths were held in a variable assigned earlier in the command (439 calls). Grep regexes and directory globs were read as secret paths. Fixing these cut the first two to 4 calls each and friction to 2.7%; held-out A results were identical before and after. The same causes reappear in the ablations: without the AST or variable resolution, friction returns. Because the fixes came from this traffic, friction is in-sample; we gave the regex an analogous tuning pass. Excluding the transcripts in which ActionGate itself was built removes 157 calls and 4 stops, leaving facts at 2.74%. A chronological split is not informative, because one project dominates the corpus. Benchmarks are written one action at a time; agents write multi-line programs.

What remains hard on held-out data. Facts' 8 misses and 4 false positives on B fall into four classes. *Open-world vocabulary* (7): destructive verbs the analyzer was not taught—Django `manage.py flush`, `prisma migrate reset`, an HTTP `DELETE` to Elasticsearch, `aws ec2 terminate-instances`, `diskutil eraseDisk`, `az group delete` and SQL inside a PowerShell cmdlet. Of held-out A's six misses on v2.1, three were of the same kind (MongoDB `deleteMany`, Redis `FLUSHALL`, `rsync --delete`) and three were structural; all six were fixed afterwards, so the current analyzer's 100% on A is not a test result. *Interpreter evaluation* (1): Perl `eval pack(...)`. *Intent modifiers* (3 false positives): `terraform plan -destroy`, `kubectl delete --dry-run`, `Remove-Item -WhatIf`. *Token-level PowerShell* (1 false positive): a quoted string in `if (Test-Path .env) {...}` taken as a command.

Lessons. First, decompose before crediting structure. Most held-out recall came from vocabulary, which a regex can

borrow; normalization added little. The syntax tree and per-tool semantics mattered where benchmarks are thin and real work is thick: in multi-line programs and in telling a read from a mention. Second, audit what an interruption is, not only how often it happens, and on traffic you did not tune on. Denials were overwhelmingly genuine reads and approvals overwhelmingly low-value. Our first disagreement audit favored facts because its guide let resolvable dynamic code count as warranted; one protocol across corpora removed most of the difference. Third, fail-closed opacity is cheap in total (77 approvals, 0.38% of our shell calls) but it is what facts add over a regex: most fact-only stops are code a careful reader, or a semantic judge, could resolve; scripts and build targets (3.1%) are a silent gap, and the analyzer is not at a fixed point: some approval false positives are fixable bugs, and fixing them is more in-sample tuning. Fourth, interpreter code is the frontier. Pattern inspection inside Python or JavaScript is matching again, and escalating all dynamic interpreter code triples friction. Together with open-world vocabulary, this argues for a two-stage design. Deterministic facts would handle the common, explainable cases, and a learned classifier or LLM judge [15], [16] only unknown tools and dynamic interpreter code.

Recommendations for deploying agent action policies.

1) *Invest first in vocabulary, then in structure.* Vocabulary carried held-out recall; a parser then interrupted less than an equally informed tokenizer, on our traffic and on other developers', although a carefully tuned regex came close.
2) *Make the unknown explicit, and know what it excludes.* Route statically unresolvable shell constructs to a human; this does not cover scripts and build targets, which need a separate control.
3) *Judge secret access by contents, not by mentions,* and consider approval rather than a hard deny for reads, which are often legitimate.
4) *Scope destructive rules by what is lost,* but expect per-call path rules to recover little: deciding whether a deletion is regenerable needs session context.
5) *Evaluate on traces you did not tune on,* report interruptions per developer or session, and audit samples of denials, approvals and disagreements. Public corpora such as SWE-chat make this possible without collecting data.
6) *Treat inline interpreter code as a separate problem,* to escalate (8.4% friction here) or delegate to a semantic judge.

VII. RELATED WORK

Agent action policies. Progent [25], AgentSpec [26], Conseca [27], ClawGuard [28] and Datalog monitors [29] enforce deterministic policies over tool calls; CaMeL [30] and Fides [31] enforce control and information flow; GuardAgent [32] and ShieldAgent [33] synthesize or verify guard logic; LlamaFirewall [34] combines regex and LLM scanners. These target mostly API-style tools [35]; a rule over a free-form shell argument remains a text rule. CARE [16] canonicalizes and parses shell commands, scores risk with weights tuned on a development split, and sends borderline cases to an LLM judge. It compares with a 22-rule regex baseline, ablates its structural layer, and reports 0.91% FPR on 329 safe commands and deny rates on benign command sets, but not interruptions on live traffic. AgentGuard [36] learns execution rules from anomalous coding-agent trajectories; we instead fix the rules and vary the predicate. Claude Code's auto mode [15] reports 0.4% FPR on 10,000 internal tool calls and 17% false negatives on 52 over-eager actions for an LLM classifier; an independent stress test found 81% false negatives on 253 ambiguous state-changing actions, 37% of which fell outside the classifier's scope as in-project file edits [37]. A per-call shell gate shares that blind spot for file edits. These figures use different datasets, labels and units and are not comparable to ours. User-level permission systems have been systematized [38], and user-authored allow/ask/never rules reduce prompts but can weaken protection [39]; our packs are expert-written, and we do not evaluate end-user authoring.

Benchmarks and containment. ToolEmu [40], R-Judge [41], RedCode [42], OpenAgentSafety [4], SABER [5] and IssueTrojanBench [6] score agents rather than gates. Real-session corpora such as SWE-chat [22] have been used to study developer-agent misalignment [43]; we use them to price gate interruptions. Sandboxes [14], OS-level enforcement [18] and recoverable filesystems [44] bound where damage lands but cannot tell `rm -rf build/` from `rm -rf src/` inside a workspace; they complement a per-call gate.

Shell analysis and obfuscation. Shell semantics are subtle [45], and static analysis of shell scripts has a long history [46]. Obfuscation frameworks show how cheaply string signatures are defeated [47]–[49]. Parsing shell commands improves learned security classification over generic tokenization [50]; we quantify the analogous effect inside a policy gate. Learned obfuscation detectors [51], [52] and AST-level deobfuscation with emulation [53], which raised VirusTotal detection of obfuscated PowerShell from 2.65% to 90%, show the value of recovering structure. We recover structure without emulating and send the unresolved remainder to a human.

Warning fatigue. Users click through some security warnings, notably SSL warnings, at high rates [11], and repeated exposure dulls neural responses to warnings [12], [13]. Agent work reduces prompts through scoped consent [54] and argues that approvals are only as honest as what they display [55]. Anthropic reports that sandboxing cut Claude Code permission prompts by 84% internally [14]. Interruption rate is therefore a first-class metric.

VIII. LIMITATIONS

Labels. All benchmarks are small and were written and labeled by models of the family that assisted development. No human or non-Claude annotator has labeled them, so the sets may share blind spots: B's 25 secret items were all caught, while the probe reviewer quickly found five unlisted credential stores. They test conformance to our ontology, not expert judgments of harm. We release a labeling kit (50 held-out B actions and the 60 out-of-sample audit items, with a guide and

an agreement script) so that others can check the agent's labels with human annotators.

Statistics. Items cluster by technique, and audit samples by session or developer, so even Holm-adjusted tests and Wilson intervals are optimistic. The one significant contrast with a tuned regex on B rests on four dynamic-code items, and B cannot resolve differences of about ten points. Out of sample, sign tests weigh a one-call difference like a large one; we therefore also report per-developer effect sizes.

Traffic. The Claude Code traffic is one developer's 15 sessions, dominated by one project, mostly on Windows, and unlabeled. It measures cost, not detection, and our audits cover samples of 30–40 calls, labeled by Claude agents and, for the approval sample, the first author; no second human labeled them. Analyzer and regexes were tuned on it, so its friction is in-sample. The Codex corpora add a second agent but not a second developer. The organic one is all PowerShell and unaudited. The Bash one is small, staged in a project and on tasks we chose, and labeled by the first author. The out-of-sample corpora add 430 developer identities and several agents (Section V), but they too are unlabeled and measure cost, and their audits are Claude agents' labels. SWE-chat's developers are commit identities in public repositories that use one checkpointing tool, a self-selected population that chose to publish its sessions and may not represent typical users; 32 of them work in that tool vendor's own repositories, and overlap with the 13 self-published developers was not checked. Every corpus records what agents issued under their own permission modes, which already prompt for some commands, and agents adapt to prompts; friction is therefore an upper bound on the extra prompts a gate adds, not its marginal cost in any one configuration.

Analysis. Labels encode a policy choice (every recursive delete is destructive). Secret detection is by file name. Beyond our ablation switches we did not test individual analyzer components, nor whether static resolution matches Bash on corner cases (ANSI-C quoting, alias or function shadowing, git aliases). The public traces contain zsh, fish and PowerShell commands that our converters pass to the Bash analyzer; 3 of the 30 out-of-sample fact-only audit items were not Bash, and such commands fail closed.

Comparisons. Tokenizer+ is our own line-based tokenizer. An off-the-shelf parser (tree-sitter-bash) reproduces the AST side, but no off-the-shelf tokenizer was compared. We compare against no LLM judge, no vendor default policy (per-user allowlists cannot be simulated offline), and no replayed incident. The probe round was one short LLM session.

Data and ethics. The in-sample traffic consists solely of the first author's own Claude Code sessions on that author's machine. Because its commands contain local paths and occasionally secrets, they are not released. The out-of-sample data are public: SWE-chat under its ODC-By license and access terms, and sessions their owners published under open licenses. We report only aggregates, number developers instead of naming them, and do not redistribute their commands; the labeling kit is regenerated from the sources. The harness reproduces every friction measurement on any user's own transcripts and prints only aggregates. Benchmark items use example domains and contain no real credentials.

IX. CONCLUSION

Holding the policy engine fixed, deriving three facts from a parsed command turns our substring policy, which stops one harmful action in six, into one that stops nine in ten. Decomposed, most of that held-out gain is vocabulary, and a regex given the same word lists comes close on a benchmark too small to separate them. Where they differ is real work. On our own traffic and, pre-specified, on 417 other developers, the fact layer interrupted much less than a tokenizer given the same semantics, here-documents and variables, chiefly on long, multi-line commands. Against the vocabulary regex it interrupted less for more developers, but the typical developer's difference is near zero, a regex tuned on our traffic interrupted about as rarely, and on 13 self-published developers both regexes interrupted less. What each interrupts for differs: regex-only stops are mostly mentions of a secret, fact-only stops mostly conservative opacity over code a careful reader can resolve, and tokenizer-only stops mostly benign. Structure buys fewer interruptions than a tokenizer, mostly by dropping spurious ones; against a regex it buys different interruptions, not clearly fewer or more valuable ones. Most remaining interruptions are approvals for deleting regenerable directories, and a per-call gate lacks the context to tell them apart. The analyzer's worst false positives surfaced only on real traffic, and a source-aware probe round located its frontier in interpreter code and unlisted credential stores. Artifact: https://github.com/muhammadusamahoyrr/actiongate.

ACKNOWLEDGMENT

M. Usama designed and built ActionGate and its analyzer, ran the evaluation and wrote the paper. A. Khalid contributed to the evaluation design, reviewed the results and the analysis of errors, and revised the manuscript. The analyzer and this paper were developed with assistance from an AI model (Claude, Anthropic). Claude-based agents also wrote the held-out and probe benchmarks, performed the second annotations, the blind audits and the joint relabeling, and produced critical internal reviews, as described in Section IV; all numbers were produced by the released harness. We thank the reviewers of earlier drafts, whose comments led to the normalized and tokenized baselines, the audits, the corrected statistics and the out-of-sample test, and the SWE-chat authors and the developers who published their own sessions.

REFERENCES

[1] S. Sharwood, "Vibe coding service Replit deleted user's production database, faked data, told fibs galore," The Register, Jul. 21, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://www.theregister.com/2025/07/21/replit_saastr_vibe_coding_incident/

[2] Amazon Web Services, "Security update for Amazon Q Developer extension for Visual Studio Code (version #1.84)," AWS Security Bulletin AWS-2025-015, Jul. 23, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://aws.amazon.com/security/security-bulletins/AWS-2025-015/

[3] M. Bar and R. McCarthy, "s1ngularity: Supply chain attack leaks secrets on GitHub: Everything you need to know," Wiz Blog, Aug. 27, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://www.wiz.io/blog/s1ngularity-supply-chain-attack

[4] S. Vijayvargiya, A. B. Soni, X. Zhou, Z. Z. Wang, N. Dziri, G. Neubig, and M. Sap, "OpenAgentSafety: A comprehensive framework for evaluating real-world AI agent safety," in *International Conference on Learning Representations (ICLR)*, 2026.

[5] Q. Hu, Y. Tang, Q. Wang, L. Zhao, P. Zhang, Y. Qing, X. Yao, D. Huang, L. Zhang, and Z. Ji, "SABER: Benchmarking operational safety of LLM coding agents in stateful project workspaces," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2026, pp. 4842–4866, arXiv:2606.01317.

[6] A. Singh, J. Yang, and T.-H. Chen, "IssueTrojanBench: Benchmarking AI coding agents against malicious issue requests," 2026, arXiv:2607.20759.

[7] Anthropic, "Configure permissions," Claude Code Documentation, accessed: Oct. 30, 2026. [Online]. Available: https://code.claude.com/docs/en/permissions

[8] ——, "Hooks reference," Claude Code Documentation, accessed: Oct. 30, 2026. [Online]. Available: https://code.claude.com/docs/en/hooks

[9] T. Claburn, "Cursor AI YOLO mode lets coding assistant run wild, security firm warns," The Register, Jul. 21, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://www.theregister.com/2025/07/21/cursor_ai_safeguards_easily_bypassed/

[10] K. Townsend, "Decades-old Bash tricks expose AI coding agents to supply chain attacks," SecurityWeek, Jun. 30, 2026, accessed: Oct. 30, 2026. [Online]. Available: https://www.securityweek.com/decades-old-bash-tricks-expose-ai-coding-agents-to-supply-chain-attacks/

[11] D. Akhawe and A. P. Felt, "Alice in Warningland: A large-scale field study of browser security warning effectiveness," in *USENIX Security Symposium*, 2013, pp. 257–272.

[12] B. B. Anderson, C. B. Kirwan, J. L. Jenkins, D. Eargle, S. Howard, and A. Vance, "How polymorphic warnings reduce habituation in the brain," in *ACM Conference on Human Factors in Computing Systems (CHI)*, 2015, pp. 2883–2892, doi: 10.1145/2702123.2702322.

[13] A. Vance, C. B. Kirwan, D. Bjornn, J. L. Jenkins, and B. B. Anderson, "What do we really know about how habituation to warnings occurs over time?" in *ACM Conference on Human Factors in Computing Systems (CHI)*, 2017, pp. 2215–2227, doi: 10.1145/3025453.3025896.

[14] D. Dworken and O. Weller-Davies, "Beyond permission prompts: Making Claude Code more secure and autonomous," Anthropic Engineering Blog, Oct. 20, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://www.anthropic.com/engineering/claude-code-sandboxing

[15] J. Hughes, "How we built Claude Code auto mode: A safer way to skip permissions," Anthropic Engineering Blog, Mar. 25, 2026, accessed: Oct. 30, 2026. [Online]. Available: https://www.anthropic.com/engineering/claude-code-auto-mode

[16] Y. Liu, W. Zhang, Z. Yang, Z. Zhang, H. Feng, X. Wang, P. Qiu, Y. Liu, B. Poczos, and J. B. Hong, "CARE: Pre-execution command verification for shell-executing LLM agents," in *IEEE International Symposium on Software Reliability Engineering (ISSRE)*, 2026, arXiv:2607.21642.

[17] X. Li, S. Yu, M. Pan, Y. Sun, B. Li, D. Song, X. Lin, and W. Shi, "Unsafer in many turns: Benchmarking and defending multi-turn safety risks in tool-using agents," 2026, arXiv:2602.13379.

[18] Y. Zheng, T. Wu, Q. Fu, T. Yu, W. Mao, T. Ma, D. Williams, W. Wang, and A. Quinn, "ActPlane: Programmable OS-level policy enforcement for agent harnesses," 2026, arXiv:2606.25189.

[19] CEL Contributors, "Common Expression Language (CEL)," Project website, accessed: Oct. 30, 2026. [Online]. Available: https://cel.dev/

[20] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. M. Wells, "Cedar: A new language for expressive, fast, safe, and analyzable authorization," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 670–697, 2024, doi: 10.1145/3649835.

[21] mvdan.cc/sh Contributors, "sh: A shell parser, formatter, and interpreter, version 3.12.0," Software, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://github.com/mvdan/sh

[22] J. Baumann, V. Padmakumar, X. Li, J. Yang, D. Yang, and S. Koyejo, "SWE-chat: Coding agent interactions from real users in the wild," in *Proc. Conf. Language Modeling (COLM)*, 2026, arXiv:2604.20779. Dataset https://huggingface.co/datasets/SALT-NLP/SWE-chat, revision 202b071f.

[23] R. G. Newcombe, "Improved confidence intervals for the difference between binomial proportions based on paired data," *Statistics in Medicine*, vol. 17, no. 22, pp. 2635–2650, 1998.

[24] ——, "Interval estimation for the difference between independent proportions: Comparison of eleven methods," *Statistics in Medicine*, vol. 17, no. 8, pp. 873–890, 1998.

[25] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," 2025, arXiv:2504.11703.

[26] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2026, arXiv:2503.18666.

[27] L. Tsai and E. Bagdasarian, "Contextual agent security: A policy for every purpose," in *Proc. Workshop on Hot Topics in Operating Systems (HotOS)*, 2025, arXiv:2501.17070.

[28] W. Zhao, Z. Li, P. Zhang, and J. Sun, "ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection," 2026, arXiv:2604.11790.

[29] N. Palumbo, S. Choudhary, J. Choi, G. Amir, P. Chalasani, I. Dillig, and S. Jha, "Formal policy enforcement for agentic systems," 2026, arXiv:2602.16708.

[30] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," 2025, arXiv:2503.18813.

[31] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," 2025, arXiv:2505.23643.

[32] Z. Xiang, L. Zheng, Y. Li, J. Hong, Q. Li, H. Xie, J. Zhang, Z. Xiong, C. Xie, C. Yang, D. Song, and B. Li, "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," in *International Conference on Machine Learning (ICML)*, 2025.

[33] Z. Chen, M. Kang, and B. Li, "ShieldAgent: Shielding agents via verifiable safety policy reasoning," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025, pp. 8313–8344.

[34] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla, A. Gampa, B. de Paola, D. Gabi, J. Crnkovich, J.-C. Testud, K. He, R. Chaturvedi, W. Zhou, and J. Saxe, "LlamaFirewall: An open source guardrail system for building secure AI agents," 2025, arXiv:2505.03574.

[35] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024, arXiv:2406.13352.

[36] W. Dai and S. Wang, "AgentGuard: Learning execution guardrails from anomalous coding-agent trajectories," 2026, arXiv:2609.16287.

[37] Z. Ji, Z. Li, P. Ma, W. Jiang, Y. Gao, and S. Wang, "Measuring the permission gate: A stress-test evaluation of Claude Code's auto mode," 2026, arXiv:2604.04978.

[38] A. E. Michael and F. Roesner, "How agents ask for permission: User permissions for AI agents, from interfaces to enforcement," in *Third Workshop on Agents in the Wild at NeurIPS*, 2026, arXiv:2607.13718.

[39] T. Yan, "Do user-authored permission policies improve protection against AI agent overreach?" 2026, arXiv:2608.27443.

[40] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024, arXiv:2309.15817.

[41] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, and G. Liu, "R-Judge: Benchmarking safety risk awareness for LLM agents," in *Findings of the Association for Computational Linguistics: EMNLP*, 2024, arXiv:2401.10019.

[42] C. Guo, X. Liu, C. Xie, A. Zhou, Y. Zeng, Z. Lin, D. Song, and B. Li, "RedCode: Risky code execution and generation benchmark for code agents," in *NeurIPS Datasets and Benchmarks Track*, 2024.

[43] N. Tang, C. Chen, G. Xu, Y. Shi, Y. Huang, C. McMillan, T. Dong, and T. J.-J. Li, "How coding agents fail their users: A large-scale analysis of developer-agent misalignment in 20,574 real-world sessions," 2026, arXiv:2605.29442.

[44] S. W. Zhong, J. Liao, J. Liu, M. Zheng, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Don't let AI agents YOLO your files: Information and control in agent-native filesystems," 2026, arXiv:2604.13536.

[45] M. Greenberg and A. J. Blatt, "Executable formal semantics for the POSIX shell," *Proceedings of the ACM on Programming Languages*, vol. 4, no. POPL, pp. 1–30, 2020, doi: 10.1145/3371111.

[46] K. Mazurak and S. Zdancewic, "ABash: Finding bugs in Bash scripts," in *Workshop on Programming Languages and Analysis for Security (PLAS)*, 2007, pp. 105–114, doi: 10.1145/1255329.1255347.
[47] D. Bohannon, "Invoke-Obfuscation: PowerShell command and script obfuscator," Software, 2016, accessed: Oct. 30, 2026. [Online]. Available: https://github.com/danielbohannon/Invoke-Obfuscation
[48] ——, "DOSfuscation: Exploring the depths of Cmd.exe obfuscation and detection techniques," FireEye, Tech. Rep., 2018.
[49] A. LeFevre, C. Barker, N. Hatfield, and E. Barker, "Bashfuscator: A fully configurable and extendable Bash obfuscation framework," Software, 2018, accessed: Oct. 30, 2026. [Online]. Available: https://github.com/Bashfuscator/Bashfuscator
[50] D. Trizna, "Shell language processing: Unix command parsing for machine learning," in *Conf. Applied Machine Learning in Information Security (CAMLIS)*, 2021, arXiv:2107.02438.
[51] D. Hendler, S. Kels, and A. Rubin, "Detecting malicious PowerShell commands using deep neural networks," in *ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2018, pp. 187–197, doi: 10.1145/3196494.3196511.
[52] V. Outrata, M. A. Polak, and M. Kopp, "Command-line obfuscation detection using small language models," 2024, arXiv:2408.02637.
[53] Z. Li, Q. A. Chen, C. Xiong, Y. Chen, T. Zhu, and H. Yang, "Effective and light-weight deobfuscation and semantic-aware attack detection for PowerShell scripts," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 1831–1847, doi: 10.1145/3319535.3363187.
[54] Y. Li, Y. Chen, P. Wang, I. Khabra, F. H. Shezan, Y. Feng, and Y. Tian, "Options, not clicks: Lattice refinement for consent-driven MCP authorization," 2026, arXiv:2605.11360.
[55] X. Weng, "What you approve is what executes: Consent integrity for black-box LLM agents," 2026, arXiv:2606.02668.